Proceedings of the ASME 2026
International Design Engineering Technical Conferences &
Computers and Information in Engineering Conference
IDETC/CIE 2026
August 23-26, 2026, Houston, TX, USA

IDETC-195131

DATA-DRIVEN KINEMATIC SYNTHESIS USING DIFFERENTIABLE PHYSICS: SAMPLING DENSITY FOR RELIABLE INITIALIZATION INTO THE BASIN OF ATTRACTION

Zhijie Lyu, Suhas Gangireddy, Anurag Purwar*
Computer-Aided Design and Innovation Lab
Department of Mechanical Engineering
Stony Brook University
Stony Brook, New York, 11794-2300

ABSTRACT

We present a retrieve-decode-refine pipeline for planar kinematic synthesis that combines nearest-neighbor retrieval over a 550,000-sample database, a LLaMA-based sequence decoder, and a decomposition-aware differentiable kinematic simulator. The simulator uses pebble-game-based decomposition to reduce the forward constraint solve to a sequence of smaller block operations, and implements this decomposition as a PyTorch computation graph so that reverse-mode automatic differentiation can supply exact refinement gradients across all seventeen planar one-degree-of-freedom topology types in the four-bar, Stephenson, and Watt families. A systematic perturbation-recovery study on 4,080 paired trials shows that AD-L-BFGS is 3.34× faster than a finite-difference Levenberg–Marquardt method and achieves a 5× precision-floor advantage at tight convergence thresholds (CD < 0.001), while both methods recover the same aggregate convergence basin ($\sigma = 0.75$), confirming that basin geometry is a property of mechanism topology rather than optimizer choice. End-to-end evaluation on ten target curves yields Excellent best-match quality (HD < 0.05) on eight queries and Good on one, with the single failure (Q8) correctly predicted by a pre-refinement $\sigma_{\text{equiv}}/\sigma_{50}$ ratio of 1.24. The best result achieves a Chamfer distance of 1.31×10^{-7} in the canonical frame. Collectively, the results establish that analytical gradients from the differentiable simulator are not merely an implementation convenience but are essential for reliable, high-precision mechanism synthesis at scale.

keywords: *Data-Driven Design, Algebraic Graph Theory, Kinematic Synthesis, Planar Mechanism, Differentiable Physics*

1 Introduction

Mechanism synthesis has long been driven by kinematic performance requirements. In many applications, the designer seeks a mechanism whose point of interest traces a prescribed curve, or whose moving link attains a sequence of desired poses. Classical path and motion synthesis methods were traditionally developed through graphical constructions, analytical formulations, and later numerical procedures specialized to low-order linkage classes [1, 2]. Exact formulations are available only for limited cases, and more general synthesis tasks typically require approximate numerical optimization [2]. At the same time, kinematic synthesis is fundamentally underdetermined: multiple distinct mechanisms may produce the same or closely similar motion, and a practically useful synthesis framework should therefore support not only feasibility, but also automated search across multiple candidate designs [3].

With the increasing demand for automation, the mechanism synthesis research has progressively shifted from classical geo-

*Address all correspondence to this author at anurag.purwar@stonybrook.edu

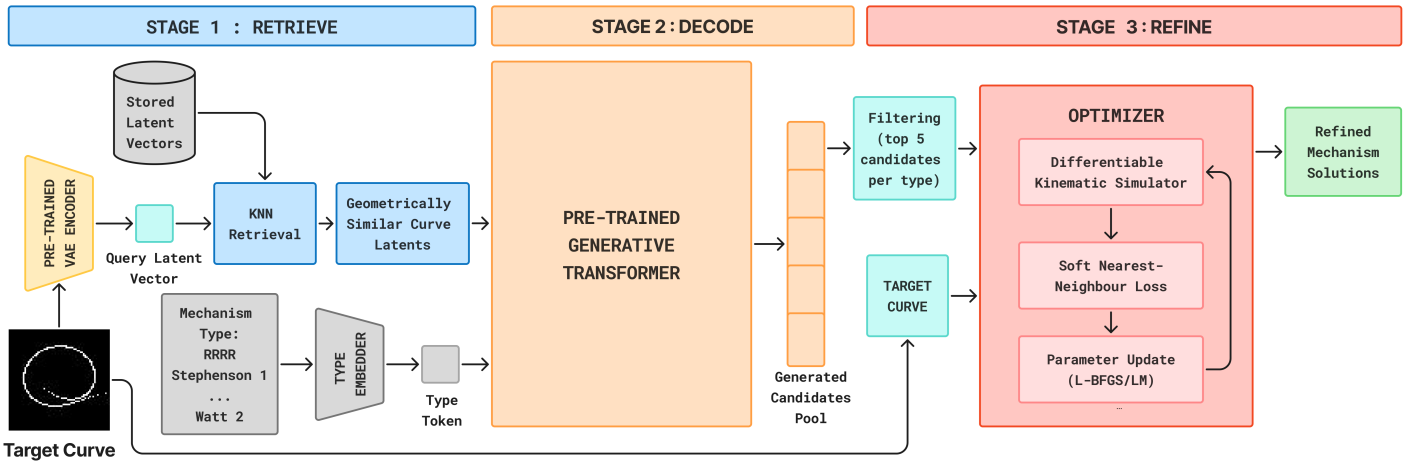

FIGURE 1: The full retrieve-decode-refine pipeline

metric and analytical approaches toward computational search and optimization [2]. Prior to the recent wave of machine learning methods, researchers had already explored a wide range of optimization-based strategies, including derivative-based formulations, derivative-free search, and more recent joint topology–geometry optimization [2, 4]. While such methods remain important, they are often computationally expensive, sensitive to initialization, and prone to local minima when the design space contains many geometrically invalid or poor-performing candidates [4].

More recently, data-driven approaches have emerged as an alternative route for automated kinematic synthesis. Broadly speaking, existing learning-based methods may be organized into four families. The first family learns a direct mapping from a curve representation to mechanism parameters or related design variables [5–12]. The second family learns a representation space for curves and mechanisms and performs neighboring-sample retrieval from a database, often followed by local refinement [13–16]. The third family uses generative models to produce candidate mechanisms, ranging from latent-space sampling methods such as variational autoencoders and conditional variational autoencoders to more recent sequence-based and autoregressive graph-generation frameworks [3,17–22]. A fourth, more distinct line formulates synthesis as a sequential decision-making problem and uses reinforcement learning to incrementally modify or grow mechanisms toward better designs [23, 24].

Despite this progress, directly predicted or generated designs often do not satisfy a user-specified custom curve with sufficient accuracy. In practice, many learning-based methods are therefore better viewed as tools for producing informative initial guesses or candidate designs rather than final solutions. For example, LInK first retrieves promising candidates in a learned embedding space and then refines them through downstream BFGS-based optimization [16]. Taken together, these developments suggest that learning and optimization are not competing paradigms, but complementary components: learning helps narrow the search space or propose structured candidates, while physics- or simulation-based numerical optimization remains important for final high-accuracy refinement.

This paper makes three contributions. **First**, we present a decomposition-aware differentiable kinematic simulator. Building on the pebble-game decomposition of [25], we map the resulting sub-problem sequence into a PyTorch computation graph, making the entire forward simulation differentiable. The simulator covers all seventeen topology types across the four-bar, Stephenson, and Watt families, whereas existing differentiable simulators are limited to Henneberg-I linkage subsets [15, 16]. **Second**, the differentiable simulator provides exact analytical gradients for parameter refinement. We demonstrate that an AD-L-BFGS optimizer using these gradients is 3.34× faster than the finite-difference Levenberg–Marquardt method and achieves a qualitative precision advantage at tight convergence thresholds that finite differences cannot reach. **Third**, we conduct a systematic perturbation-recovery study across all seventeen topologies to characterize the convergence basin of the refinement stage. The central finding is that the basin geometry is a property of mechanism topology, not of the optimizer. We further introduce the $\sigma_{\text{equiv}}/\sigma_{50}$ criterion, which translates a decoded candidate's pre-refinement curve distance into an equivalent perturbation magnitude, providing a practical diagnostic of whether refinement will succeed before it is run.

The retrieve-decode-refine pipeline is the authors' own design. It uses a pre-trained VAE encoder [26, 27] as a feature extractor for the kNN retrieval step, and a pre-trained LLaMA-based sequence decoder [28–30] as the candidate generator. The kNN retrieval step, the full pipeline architecture, and the mech-

anism database are the authors' own contributions [14, 31]; only the pre-trained decoder model weights are drawn from prior work.

The paper is organized as follows. Section 2 describes the three-stage pipeline architecture and the role of each component. Section 3 presents the pebble-game decomposition, the DAG-structured computation graph, and the forward-simulation and reverse-mode refinement procedures. Section 4 reports two experiments: Section 4.1 establishes the convergence basin as a function of mechanism topology across 4,080 paired trials, and Section 4.2 verifies that the basin-study predictions transfer to end-to-end pipeline performance on ten target queries. Section 5 summarizes the findings and outlines directions for future work.

2 Overview of Pipeline

The proposed framework addresses planar mechanism synthesis as a three-stage retrieve-decode-refine pipeline. Given an arbitrary target coupler curve C^* supplied by the designer, the system (1) retrieves the nearest precomputed embeddings from a large offline database, (2) decodes the retrieved embeddings into candidate mechanism parameter vectors using a generative model, and (3) refines the most promising candidates through a differentiable kinematic optimizer to produce a precise solution. Each stage is described in detail below, and Fig. 1 shows an overview of the entire process.

2.1 Embedding Retrieval

A database of approximately 550,000 precomputed mechanism samples and their corresponding coupler curve embeddings is maintained offline. Each curve is encoded into a 50-dimensional latent vector using the mean (μ) output of a pre-trained Variational Autoencoder (VAE) [26, 27], which provides a compact continuous representation of curve shape. At query time, the target curve is passed through the same frozen VAE encoder to produce a query latent vector. A k-nearest neighbor (kNN) search over the latent database ($k = 50$) identifies the nearest latent vectors under the Euclidean metric. This retrieval step narrows the search space from half a million entries to fifty promising neighbors in seconds. Details of the VAE training and database preprocessing can be found in [14, 31].

2.2 Generative Decoding via the LLaMA Model

For each of the fifty retrieved neighbors, a pre-trained sequence generation model adapted from the LLaMA transformer architecture [28, 29] generates candidate mechanism parameter vectors. The model accepts two conditioning inputs: (1) the VAE latent vector of the retrieved curve and (2) a topology class label identifying which of the seventeen supported mechanism types to generate, spanning the four-bar, Stephenson, and Watt linkage families. The model generates mechanism joint coordinates sequentially—each coordinate predicted from those already generated—yielding a complete candidate parameter vector. For the architectural details and training setup of the decoder, see [30].

Decoding is performed across all seventeen supported topology classes for every retrieved neighbor, yielding up to $50 \times 17 = 850$ candidate parameter vectors per query. These candidates are ranked by Chamfer distance to the target, and the top 5 candidates per topology (85 candidates in total) are passed to the refinement stage as a topology-diverse pool of initializations. This scope is used for both experiments reported in Section 4: the convergence-basin study spans all seventeen topologies, and the end-to-end pipeline evaluation is performed on ten target curves spanning five mechanism families.

2.3 Differentiable Kinematic Refinement

The 85 candidates are refined locally using one of two optimizer variants. The first variant, **FD-LM**, applies Levenberg–Marquardt (LM) with a finite-difference (FD) Jacobian, which iterates simulation in the configuration space with small changes and returns an approximated Jacobian. The second variant, **AD-L-BFGS**, uses the PyTorch-differentiable interface of the simulator to compute exact gradients via reverse-mode automatic differentiation (AD) in a single backward pass, and then uses the L-BFGS algorithm for optimization [32].

Both variants employ a soft nearest-neighbor loss [33] in place of the hard Chamfer distance. The standard Chamfer distance assigns each predicted curve point to its single nearest target point, which creates a non-differentiable, piecewise-linear objective. The soft variant instead computes a weighted sum over all target points, with weights governed by a temperature parameter τ (annealed from 0.5 to 0.01). As τ decreases, the weighted sum sharpens toward the hard Chamfer limit; the annealing schedule keeps the objective smoothly differentiable throughout optimization. The best solution across all refined candidates, tracked by hard Chamfer distance, is returned as the final pipeline output.

3 Differentiable Simulator

We have developed a differentiable kinematic simulator that combines pebble-game-based constraint decomposition with PyTorch's automatic differentiation. The decomposition reduces the forward constraint solve to smaller block operations; recording this computation as a PyTorch graph then enables exact reverse-mode gradients for the refinement stage. This section presents the decomposition strategy, the resulting computation graph, and the numerical procedure.

3.1 Review of the Pebble Game Algorithm

Rooted in Laman's theorem [34], the pebble game algorithm is an efficient method for analyzing a constraint graph, with a time complexity of $O(|n|^2)$ [25], where n is the number of joints. The algorithm was initially introduced in the study of amorphous planar materials [35], and was later refined by the rigidity theory community [36]. In mechanical engineering, Shai et al. presented a theoretical study on the mobility of linkages using this algorithm [37].

While Laman proposed a mobility-count-based criterion originally intended for skeletal structures [34], the mechanical engineering community has long used Grübler's formula to estimate the mobility of mechanisms [38]. These two viewpoints can be regarded as conceptually equivalent in terms of mobility counting when kinematic constraints are represented using the point-based model [39]. To the best of our knowledge, however, combinatorial mobility determination methods rooted in Grübler's criterion are generally exponential or factorial in complexity, whereas methods based on Laman's theorem admit polynomial-time formulations [40, 41]. The mobility analysis time difference becomes significant when the number of variables exceeds roughly twenty.

3.2 From Constraint Graph to Computation Graph

From our previous work [25], we observe that the numerical analysis of a linkage graph can be organized as a sequence of directed acyclic graphs (DAGs) [42] after an appropriate condensation procedure [43]. This property is important for computation because artificial neural networks (ANNs) are also DAG-structured, and modern machine learning libraries such as PyTorch [44] and JAX [45] explicitly construct and store computation graphs. Once the same computation flow is formulated in this manner, these libraries can be used directly to perform forward simulation, record the computational process, and apply reverse-mode automatic differentiation (AD) to fine-tune parameters toward desired kinematic performance.

This idea is inspired by the work of Nobari et al. [15, 16], who developed a JAX-based differentiable simulator for linkages. However, while in theory all minimally rigid graphs can be generated through the combination of two types of Henneberg moves [46], their linkage graphs are primarily generated using Henneberg-I moves, resulting in a limited range of linkage types. In contrast, the constraint graph constructed in our framework requires an additional condensation step to become directed and acyclic, but can accommodate arbitrary combinations of planar geometric constraints.

We also note that computation graphs for linkage simulation are often substantially deeper than those of typical ANNs: they may contain hundreds to hundreds of thousands of mostly linear computational layers, whereas ANNs usually contain tens to hundreds of layers, many of which are nonlinear [47–49]. Nevertheless, as the experiments show, the resulting gradients do not vanish in practice and remain effective for tuning kinematic configurations.

3.3 Explanation of the Numerical Procedure

This subsection briefly outlines the forward simulation and reverse-mode refinement procedures that underlie the decomposition-aware differentiable simulator. For any kinematic constraint system Φ of multiple constraint expressions at time step t, we want to find $\mathbf{q}_t$ such that,

$$\Phi_t = [f_i(\mathbf{q}_t, t)] = [\mathbf{0}], \tag{1}$$

where $\mathbf{q}_t$ is the set of parameters that determines the geometric system. In the first part of the forward stage, i.e., the simulation stage, its Jacobian matrix is

$$J_t = \frac{\partial \Phi_t}{\partial \mathbf{q}_t}. \tag{2}$$

We can solve this using the following iteration [50]:

$$\mathbf{q}_t \leftarrow \mathbf{q}_t - (J_t^T J_t + \lambda I)^{-1} J_t [f_i(\mathbf{q}_t, t)] \tag{3}$$

Then, when the simulation is complete, we stack all computed parameters into a compact tensor. In the point-based model, the shape of $\mathbf{q}_t$ is $(n, 2)$, and we compute T number of states. Therefore, we stack all time steps along one dimension, and we get a data object $\mathbf{P}$ of shape $(n, 2, T)$. The sequence for the coupler point p_c is saved in a slice of this tensor, which can be denoted as $\mathbf{P}[c]$. We can consider $\mathbf{P}[c]$ as the *performance* of the linkage mechanism, if the coupler curve matching is the only design criterion. We expect these points to match the sequence $\mathbf{P}_{exp}$, so we can define the error e as follows:

$$e = ||\mathbf{P}[c] - \mathbf{P}_{exp}||_2^2, \tag{4}$$

which is the sum of squared errors of multiple data points. Other performance types, such as poses, can be computed with an extra layer that uses two points on a particular linkage object to define the pose. In essence, a general error e_g can be expressed as the sum of the squares of the residuals r_j:

$$e_g = \sum_j r_j^2 = \sum_j ||(\text{performance}_j(\mathbf{P}) - \text{expectation}_j)||_2^2. \tag{5}$$

A differentiable computation graph can be generated, where e_g is the end vertex of this DAG, while the initial state $\mathbf{q}_0$ includes

all starting vertices. Updating the initial $\mathbf{q}_0$ results in the change of the kinematic configuration, and iteratively updating $\mathbf{q}_0$ is the *fine-tuning* process, which requires reverse-mode AD. We can first compute the Jacobian for reverse mode:

$$J_{RMAD} = \frac{\partial [r_j]}{\partial \mathbf{q}_0}, \tag{6}$$

and use it alongside well-established derivative-based optimization algorithms such as BFGS [32]. The computation process is recorded automatically by modern computing architectures and libraries. Recent works such as [16, 21] have used the same computational architecture for kinematic synthesis.

We should note that solving a geometric system without decomposing the Jacobian matrix can be computationally expensive, because an $n \times n$ matrix operation is generally $O(n^3)$. Additionally, this is unnecessary. Shai et al. presented a study on the Jacobian matrix, which shows that when the residuals f_is are sorted appropriately, the Jacobian matrix is a lower block-triangular matrix, whose upper right part elements are zeros [37]. These blocks are decomposable, which is equivalent to the decomposition procedure shown in the pebble game algorithm. Therefore, we can use these smaller blocks, which are also Jacobian matrices, for computation. For a block of 2×2, we can use dyadic decomposition ($O(1)$ for each block) to speed up the process even further.

4 Experimental Evaluation

We evaluate the proposed framework in two stages. First, a population-scale convergence-basin study (Section 4.1) measures the refinement landscape across all 17 mechanism topologies, compares the two optimization backends on 4,080 paired trials, and evaluates mechanism topology in terms of basin geometry. Second, end-to-end pipeline runs (Section 4.2) verify that the AD-L-BFGS refiner produces high-quality synthesis results across multiple mechanism families when initialized by the retrieve-decode stage.

Positioning with respect to prior work. The pipeline architecture—retrieval in a learned embedding space followed by parameter refinement—is closest to the LInK family of methods [15, 16], which also train an encoder to map similar curves to nearby positions in a shared embedding space, and then refine retrieved candidates using BFGS. The present work differs in two respects: (1) refinement is performed by a decomposition-aware differentiable simulator that supports arbitrary planar one-degree-of-freedom topologies rather than a fixed Henneberg-I subset, and (2) the experimental emphasis is on the refinement landscape itself—the convergence basin as a function of topology—rather than on end-to-end retrieval accuracy alone. Recent sequence-based and diffusion-based generators from MechaFormer [21] and LinkD [22] can be read as alternative decoders in stage (2) of the pipeline; a direct quantitative comparison is not performed here because their released models do not span the full set of seventeen topologies covered by our basin study. The random-initialization baseline reported in Section 4.2 provides an external reference point that is independent of any particular learned decoder.

TABLE 1: Topology classes and structural features. The 17 types span five-, seven-, and eight-joint mechanisms from five kinematic families (four-bar, Stephenson-I, Stephenson-III, Watt-I, and Watt-II).

Type	Family	Joints	Param Dim
RRRR	4-bar	5	10
Steph1T1	Stephenson-I	8	16
Steph1T2	Stephenson-I	8	16
Steph1T3	Stephenson-I	8	16
Steph3T1A1	Stephenson-III	7	14
Steph3T1A2	Stephenson-III	7	14
Steph3T2A1	Stephenson-III	8	16
Watt1T1A1	Watt-I	8	16
Watt1T1A2	Watt-I	8	16
Watt1T2A1	Watt-I	8	16
Watt1T2A2	Watt-I	8	16
Watt1T3A1	Watt-I	7	14
Watt1T3A2	Watt-I	7	14
Watt2T1A1	Watt-II	8	16
Watt2T1A2	Watt-II	8	16
Watt2T2A1	Watt-II	8	16
Watt2T2A2	Watt-II	8	16

4.1 Convergence Basin Study

4.1.1 Experimental Setup We construct a paired benchmark spanning the full mechanism database, with joint coordinates ranging $[-10, 10]$. For each of 17 topologies (Table 1), we draw 20 ground-truth instances and perturb the joint-coordinate vector with isotropic Gaussian noise at twelve magnitudes:

$\sigma \in \{0.05, 0.10, 0.20, 0.30, 0.50, 0.75, 1.00, 1.25, 1.50, 2.00, 2.50, 3.00\}$.

Both **FD-LM** and **AD-L-BFGS** are launched from the identical perturbed initialization on every (topology, instance, σ) triple, yielding $17 \times 20 \times 12 = 4{,}080$ paired trials per method (8,160 total). Recovery success is defined as final Chamfer distance (CD) below 0.1, and the convergence basin radius is defined as the largest σ at which the aggregate success rate remains above 50%.

4.1.2 Both Methods Recover the Same Basin

The central finding is that both optimizers produce essentially the same convergence basin. Both **FD-LM** and **AD-L-BFGS** yield an aggregate basin radius of $\sigma_{\text{basin}} = 0.75$. Fig. 2(a) shows the aggregate success curves; they are nearly identical at every σ level. Raw success rates (dots) are accompanied by 95% bootstrap confidence bands. To obtain a continuous estimate of the 50%-success perturbation magnitude, we fit a standard logistic regression to the binary trial outcomes. The resulting smooth S-curves (solid lines) cross the 50% threshold at $\sigma_{50} \approx 1.15$ (FD-LM) and $\sigma_{50} \approx 1.19$ (AD-L-BFGS), marked by diamond symbols. For reference, the largest discrete test point at which the raw aggregate success rate remains above 50% is $\sigma = 0.75$ (dotted line); the logistic fit interpolates a more precise crossing from the full trial data.

Breaking down by topology (Fig. 2(b)), per-type basin widths range from 0.77 to 1.79. Because each width is estimated from a finite number of trials, we accompany it with a 95% confidence interval (CI)—a range within which the true basin width is expected to lie, accounting for sampling variability. The CIs of the two methods overlap on every type, meaning the observed differences are within statistical noise. The takeaway is direct: basin shape is a property of the mechanism topology, not of the optimizer. Both methods are limited by the same underlying landscape geometry.

Table 2 reports aggregate success rates at three CD thresholds. The gap between methods widens at tighter thresholds, where the quality of analytical gradients becomes more important.

TABLE 2: Multi-threshold success rates on 4,080 paired trials per method. The gap between methods widens at tighter thresholds.

Threshold	FD-LM	AD-L-BFGS
CD < 0.1	53.6% (2,187)	54.5% (2,223)
CD < 0.01	28.8% (1,176)	31.7% (1,295)
CD < 0.001	5.5% (224)	8.8% (358)

4.1.3 AD-L-BFGS Is 3.34× Faster

The two methods reach the same basin, but at very different computational costs. Since every trial is paired, both methods start from the identical perturbed initialization, so we can directly compare their wall times on the same problem instance. Fig. 3 reports the geometric mean speedup ratio (**FD-LM** wall time / **AD-L-BFGS** wall time) for each topology, along with 95% bootstrap confidence intervals (CIs) [51].

The aggregate speedup is 3.34× (95% CI [3.27, 3.42]), and every topology is strictly above 1×, ranging from 2.46× (**Watt1T3A1**) to 6.11× (**Steph3T1A2**). The speedup comes from fewer iterations, not cheaper iterations. Per-iteration wall times differ only modestly between the two methods (e.g., Steph1T1: 5.58 s/iter for **FD-LM** vs. 4.58 s/iter for **AD-L-BFGS**, a 22% difference per iteration). The large aggregate speedup arises because **AD-L-BFGS** converges in a median of 26 iterations versus 102 for **FD-LM**. This is because L-BFGS uses gradient history to approximate curvature.

At high precision, the advantage becomes qualitative. Fig. 4 decomposes the 4,080 paired trials into four outcomes at three CD thresholds. At CD < 0.1, the two methods are nearly interchangeable: 50.2% of trials are solved by both, with small margins of AD-only (4.3%) and FD-only (3.4%) successes. At CD < 0.001, the picture changes sharply: **AD-L-BFGS** succeeds on 4.2% of trials where **FD-LM** cannot, versus only 0.9% in the reverse direction—roughly a 5× precision-floor advantage. For applications requiring high-precision path matching, BFGS is not merely faster but enables a level of precision the FD method cannot reach.

4.1.4 Failure Analysis

Of the 4,080 trials per method, failure counts are nearly identical: **FD-LM** fails on 1,893 trials (46.4%) and **AD-L-BFGS** on 1,857 (45.5%), reinforcing that both methods are limited by the same landscape geometry. In both cases, the dominant failure mode is stagnation—the optimizer reaches a loss plateau and cannot reduce CD further. Importantly, we observe zero local-minimum traps across all 8,160 trials: no case where the initial CD was below 0.1 but the optimizer diverged to a worse solution. Failures are entirely due to starting too far from the true solution.

4.2 End-to-End Pipeline Evaluation

Having characterized the refinement landscape, we evaluate the full retrieve-decode-refine pipeline on ten target curves drawn from five mechanism families. The pipeline follows the architecture described in Section 2, with refinement capped at 200 **AD-L-BFGS** steps per candidate. The purpose of this experiment is to verify that the convergence-basin predictions of Section 4.1 transfer to end-to-end pipeline use: specifically, that queries whose decoded candidates fall within the measured basin succeed, and queries whose candidates fall outside the basin fail.

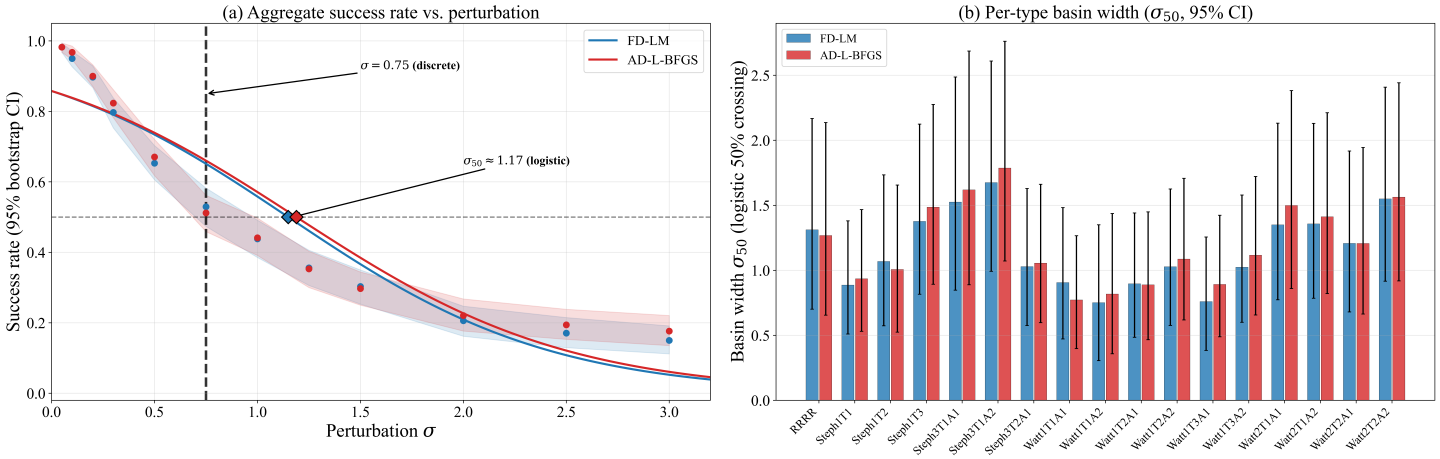

FIGURE 2: Convergence basin analysis. (a) Aggregate success rate vs. perturbation magnitude. Dots: raw empirical rates; solid curves: logistic fits; shaded bands: 95% bootstrap CIs. The dotted line marks the discrete basin radius $\sigma = 0.75$; diamond markers indicate the logistic 50%-crossing $\sigma_{50} \approx 1.17$. Both **FD-LM** and **AD-L-BFGS** trace near-identical profiles. (b) Per-type basin width (logistic 50% crossing with 95% CI) for all 17 types. Widest basins: **Steph3T1A2** (1.79) and **Steph3T1A1** (1.62); narrowest: **Watt1T1A2** (0.82) and **Watt1T1A1** (0.77).

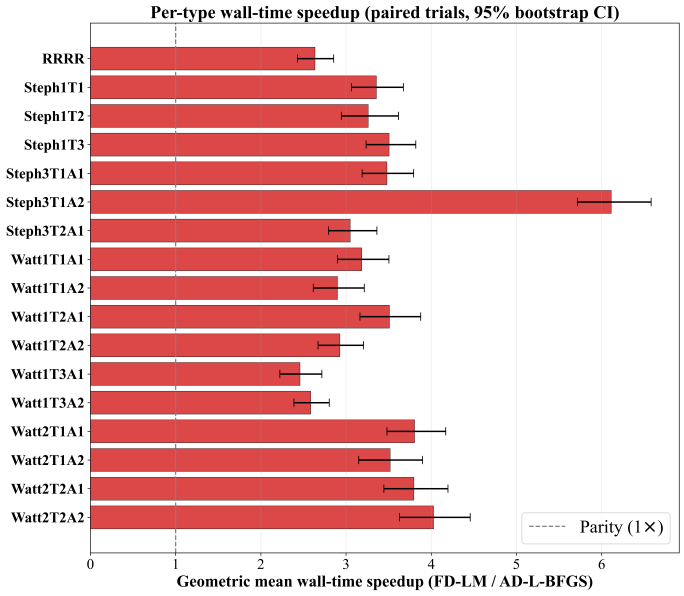

FIGURE 3: Per-type wall-time speedup of **AD-L-BFGS** over **FD-LM** (geometric mean with 95% bootstrap CIs). Every topology is strictly above 1×. Aggregate: 3.34× [3.27, 3.42]. Range: 2.46× (**Watt1T3A1**) to 6.11× (**Steph3T1A2**).

The ten queries constitute a focused validation set, not a comprehensive benchmark.

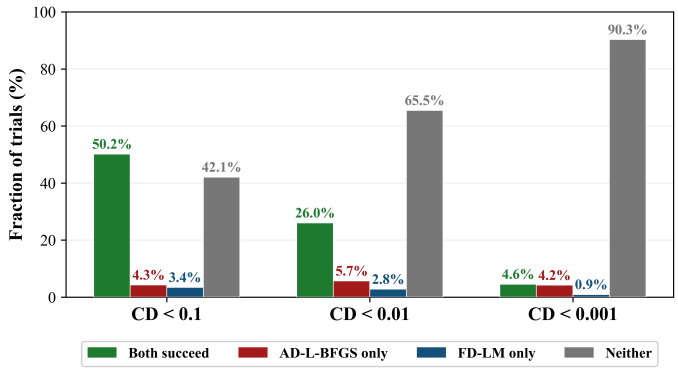

FIGURE 4: Capability gap at three CD thresholds (both optimizers run to a fixed budget of 200 iterations). At CD < 0.1 the two methods show similar success counts (AD-only 4.3%, FD-only 3.4%). At CD < 0.001 the gap widens sharply: AD-only 4.2% vs. FD-only 0.9%, a 5× precision-floor advantage for **AD-L-BFGS**.

Evaluation metric. Curve quality is measured in a canonical frame—each curve is centered at its centroid, scaled to unit bounding radius, and PCA-aligned—which removes placement, scale, and orientation differences that are irrelevant to shape comparison [27]. We report two complementary distance metrics. *Chamfer distance* (CD) measures global shape agreement: for each point on one curve, it finds the nearest point on the other curve, squares that distance, and averages over all points in both directions. A low CD means the curves agree closely

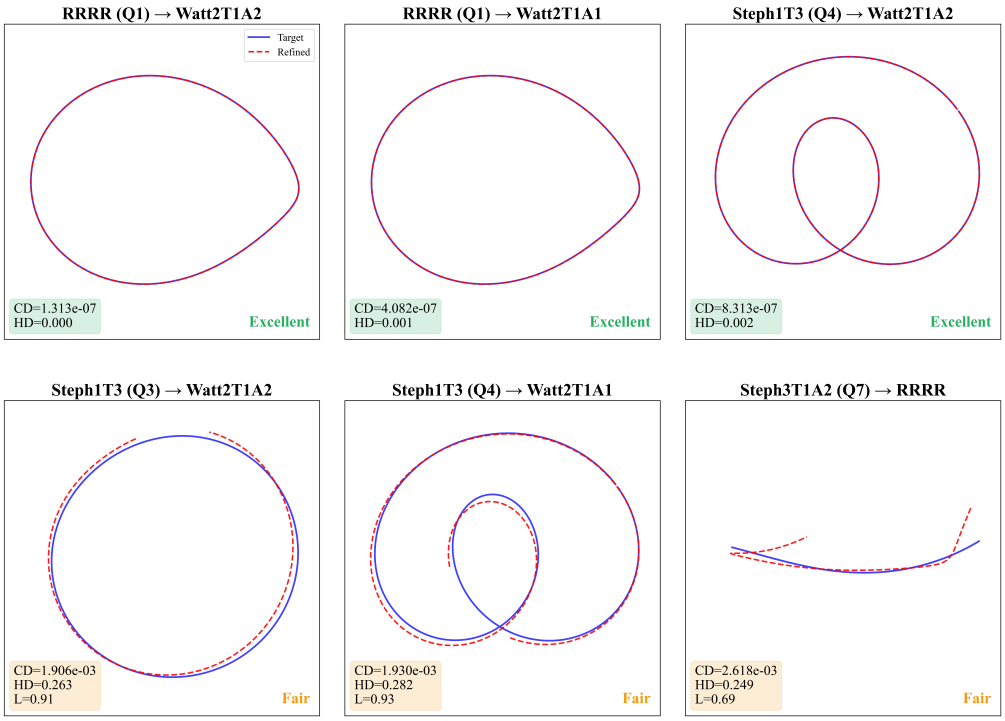

FIGURE 5: Top row: candidates with low CD *and* low HD—genuinely good geometric matches. Bottom row: candidates with similarly low CD but HD > 0.20, where the refined curve misses a local feature of the target.

on average. *Hausdorff distance* (HD) [52] measures worst-case disagreement: it finds the single largest nearest-point gap between the two curves. A low HD guarantees that no part of the curve deviates beyond an acceptable limit. Together, CD summarizes global shape agreement while HD guards against localized defects—a curve that matches the target everywhere except at one cusp or loop will have low CD but high HD (Fig. 5).

After canonicalization, curves fit inside a unit-radius circle. Quality labels are assigned based on HD relative to this scale: **Excellent** (HD < 0.05, within 5%), **Good** (HD < 0.15, within 15%), **Fair** (HD < 0.3, within 30%), **Poor** (otherwise). We also report the arc-length ratio $L = \text{arc(refined)}/\text{arc(target)}$, which catches a class of error that neither CD nor HD detects: if the refined curve traces an extra loop or skips part of the target path, the nearest-point distances may still be small, but L will deviate from 1. Candidates with anomalous L values are excluded from best-match selection in Table 3.

4.2.1 Pipeline Results

Across the ten queries, Q1–Q7, Q9, and Q10 succeed—eight at Excellent quality and one (Q6) at Good—while Q8 fails entirely (Table 3). Representative synthesized curves are shown in Fig. 6. Results are discussed below by ground truth mechanism family.

4-bar. Q1 and Q2 both reach Excellent. Q1 is the strongest result across all ten queries: the best refined curve is geometrically indistinguishable from the target ($\text{CD} = 1.31 \times 10^{-7}$, HD $<$ 0.001), and 36 of 85 candidates are Excellent. Fig. 7 shows six distinct topology types all reproducing the same coupler curve at Excellent quality, demonstrating cross-topology synthesis from a single query. Only Q1 is shown in Fig. 7; Q2 yields consistent results with 31/85 Excellent candidates.

Stephenson-I. Four queries span three Stephenson-I topology types (Q3–Q6). Both Steph1T3 queries (Q3, Q4) reach Excellent with large pools of high-quality candidates (69/85 and 29/85 Good-or-better respectively). The best result for Q3 ($\text{CD} = 1.37 \times 10^{-5}$, $\text{HD} = 0.007$) is achieved by a Watt2T1A1 cross-topology decode, confirming that topologically distinct mechanisms can reproduce the same coupler curve. Steph1T1 (Q5) also reaches Excellent ($\text{CD} = 4.28 \times 10^{-6}$, $\text{HD} = 0.003$; 29/85 Good-or-better). Steph1T2 (Q6) is the weakest success: the best candidate reaches Good ($\text{CD} = 2.77 \times 10^{-3}$, $\text{HD} = 0.100$) but not Excellent, with only 1 of 85 candidates clearing the Good threshold. As shown in Section 4.2.2, this query's decoded candidates start near the edge of the convergence basin ($\sigma_{\text{equiv}}/\sigma_{50} = 0.88$), consistent with the marginal outcome.

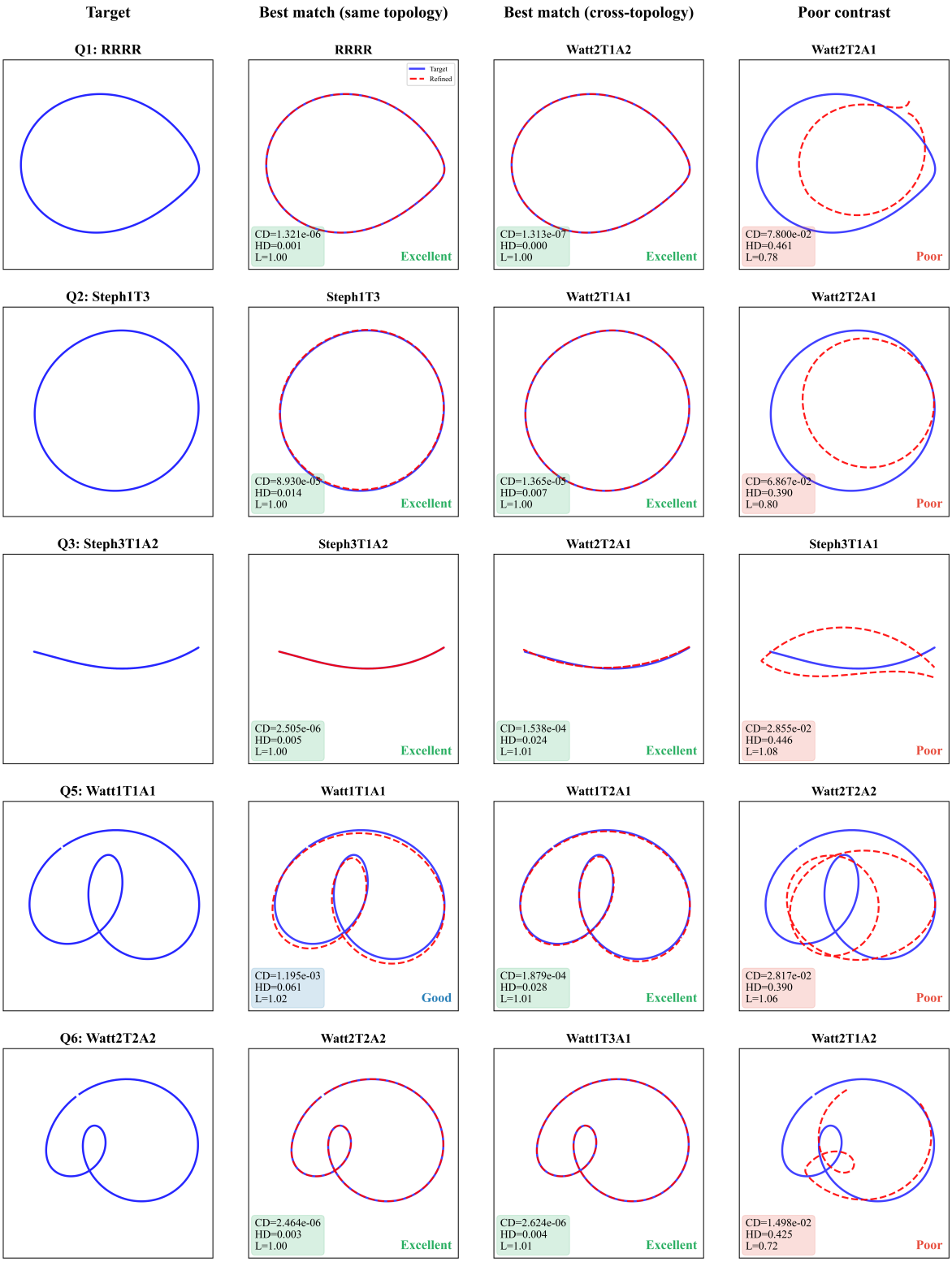

FIGURE 6: End-to-end pipeline results in the **canonical frame**. Each row shows one representative successful query per mechanism family (4-bar, Stephenson-I, Stephenson-III, Watt-I, Watt-II), selected from the ten queries in Table 3. Columns: (1) target coupler curve, (2) best same-topology refined match, (3) best cross-topology refined match, (4) a Poor-quality contrast example. Overlays show the target (solid blue) and the refined candidate (dashed red). Quality labels use CD, HD, and arc-length ratio L.

TABLE 3: End-to-end pipeline results across ten queries. All refinement uses **AD-L-BFGS**. For each query, 50 retrieved neighbors are decoded across all 17 topology types (850 candidates); the top 5 per topology ranked by Chamfer distance are kept, giving 85 refined candidates per query. "GT Family" is the mechanism family of the target; "Query Type" is its specific kinematic topology. "Best CD" and "Best HD" are measured in the canonical frame on the best arc-length-valid candidate ($L \in [0.85, 1.20]$). E, G, F, P give the counts of Excellent, Good, Fair, Poor candidates out of 85. Three topology types (**RRRR**, **Steph1T3**, **Watt1T1A1**) are each queried with two different target instances.

Q#	GT Family	GT Type	Best CD	Best HD	Quality	E	G	F	P
1	4-bar	RRRR	1.31×10^{-7}	0.000	Excellent	36	8	14	27
2	4-bar	RRRR	2.07×10^{-6}	0.002	Excellent	31	7	20	27
3	Stephenson-I	Steph1T3	1.37×10^{-5}	0.007	Excellent	35	34	9	7
4	Stephenson-I	Steph1T3	8.31×10^{-7}	0.002	Excellent	25	4	28	28
5	Stephenson-I	Steph1T1	4.28×10^{-6}	0.003	Excellent	3	26	44	12
6	Stephenson-I	Steph1T2	2.77×10^{-3}	0.100	Good	0	1	22	62
7	Stephenson-III	Steph3T1A2	2.51×10^{-6}	0.005	Excellent	9	41	30	5
8	Watt-I	Watt1T1A1	1.03×10^{-2}	0.378	Poor	0	0	0	85
9	Watt-I	Watt1T1A1	1.88×10^{-4}	0.028	Excellent	11	9	15	50
10	Watt-II	Watt2T2A2	2.46×10^{-6}	0.003	Excellent	27	16	25	17

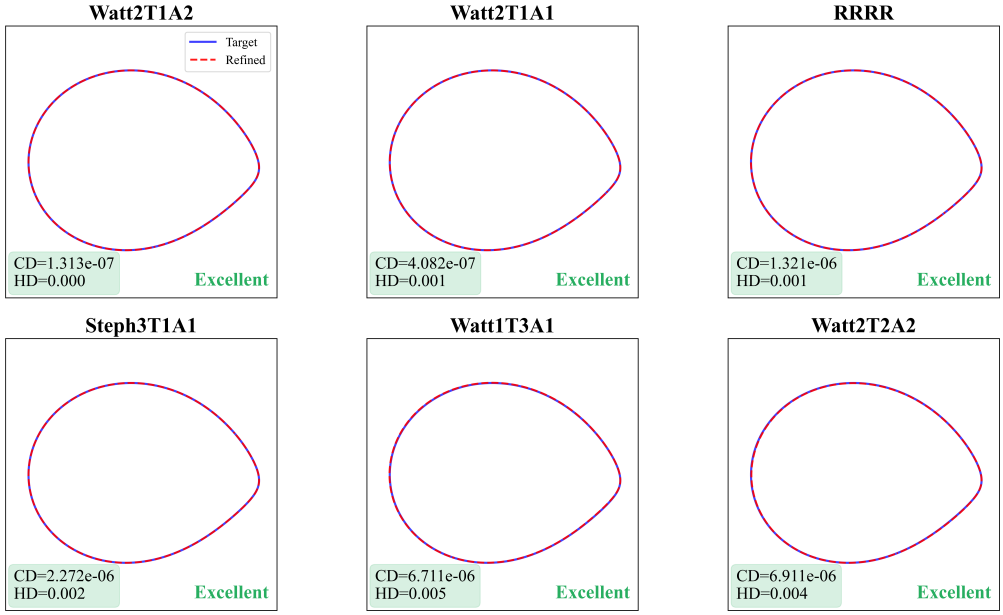

FIGURE 7: Cross-topology synthesis on the 4-bar query (Q1). Six distinct mechanism topologies (dashed red) all reproduce the 4-bar target curve (solid blue) at Excellent quality (HD < 0.05). The pipeline finds the same shape through structurally different linkages.

Stephenson-III. The single query (Q7) reaches Excellent, with the best candidate matching the correct query type itself (Steph3T1A2, CD $= 2.51 \times 10^{-6}$, HD $= 0.005$), demonstrating accurate type identification. 50/85 candidates reach Good-or-Excellent quality.

Watt-I. The same topology type (Watt1T1A1) is queried with two different target curves, producing opposite outcomes. Q8 fails entirely: all 85 candidates remain Poor (best HD $= 0.378$). Q9 succeeds: the best candidate reaches Excellent (CD $= 1.88 \times 10^{-4}$, HD $= 0.028$) with 20/85 Good-or-better. This divergence

within the same mechanism family is the clearest evidence that pipeline success depends on initialization quality rather than mechanism topology. Section 4.2.3 traces the cause to a large gap in decoded-candidate quality between the two queries.

Watt-II. The single query (Q10) reaches Excellent (CD $= 2.46 \times 10^{-6}$, HD $= 0.003$) with 43/85 Good-or-better candidates. The best candidate matches the correct query type (Watt2T2A2).

Why a multi-metric evaluation is necessary. Fig. 5 shows why a single-metric evaluation is insufficient. The top row shows genuine high-quality matches (low CD *and* low HD); the bottom row shows candidates with equally small CD but much larger HD. Across the ten queries, 325 of the 850 refined candidates satisfy CD < 0.05 but have HD > 0.20, meaning a CD-only table would overcount matches by roughly 38%. The arc-length ratio L catches a third class of artifact: in the Stephenson-III query, a Watt1T2A2 cross-topology decode achieves CD $= 1.61 \times 10^{-4}$ and HD $= 0.042$ (both within Excellent thresholds) but has $L = 0.57$, meaning the refined curve covers only about half the target arc length. Together, CD, HD, and L form a three-metric criterion that guards against global drift, local defects, and topological artifacts, respectively.

Comparison with random initialization. To quantify the value of the retrieve-decode stage, we compare against a random-init baseline: for each family, 60 candidates are drawn from a uniform distribution over the per-type bounding box and refined with the same **AD-L-BFGS** optimizer. Table 4 reports the optimizer's Chamfer distance in the same raw coordinate frame used in the basin study (Section 4.1), not the canonical-frame CD reported in Table 3, so that the random-baseline success criterion (CD < 0.1) is directly comparable to the basin study's convergence threshold. Random initialization achieves a 1.7% aggregate success rate (5/300) across all five families, compared to the pipeline's 24–81% rate of reaching Good-or-better quality across eight of nine successful queries (Table 3); the ninth, Q6, is a marginal success with a single Good candidate out of 85 (1.2%). Even the weakest pipeline success (Q6, Good) produces a match that random initialization cannot. The retrieve-decode stage is not optional: it is the differentiable simulator's force multiplier.

4.2.2 The Basin Study Predicts Pipeline Outcomes

The basin study (Section 4.1) establishes how much perturbation each mechanism type can tolerate before the optimizer fails to converge. This section connects those findings to the pipeline by estimating how far each pipeline candidate is from the target, expressed as a perturbation magnitude comparable to those used in the basin study.

The bridge works as follows. In the basin study, each trial applies a known perturbation σ to the joint coordinates and produces a perturbed curve. For each trial, we also compute the canonical-frame CD between the perturbed and ground-truth curves, giving a per-type relationship between perturbation magnitude and curve deviation (Fig. 8(a)). This relationship can be read in reverse: given a pipeline candidate's canonical CD *before* refinement, we look up which perturbation magnitude σ would produce the same amount of curve deviation. We call this σ_{equiv}.

The final comparison is entirely in perturbation-magnitude space: σ_{equiv} (how far the pipeline candidate effectively is from the target) versus σ_{50} (the perturbation level at which half of basin-study trials converge). If $\sigma_{\text{equiv}} < \sigma_{50}$, the candidate starts within the convergence basin and refinement should succeed. If $\sigma_{\text{equiv}} > \sigma_{50}$, failure is expected. Fig. 8(b) shows the ratio $\sigma_{\text{equiv}}/\sigma_{50}$ for each pipeline query: values below 1 (green) predict success, values above 1 (red) predict failure.

The prediction is correct for all ten queries (10/10). The nine successful queries have $\sigma_{\text{equiv}}/\sigma_{50}$ ratios between 0.004 and 0.88—their decoded candidates start inside the convergence basin. Q8 has $\sigma_{\text{equiv}}/\sigma_{50} = 1.24$, placing its candidate outside the basin, and the optimizer cannot recover. The gap between the closest success (Q6 at 0.88) and the failure (1.24) indicates a meaningful separation. The two Watt-I queries (Q8, Q9) provide a direct within-type comparison: Q9 lands at 0.58 (inside the basin, success) while Q8 lands at 1.24 (outside, failure), confirming that the basin boundary is the operative threshold.

TABLE 4: Pipeline initialization vs. random initialization. Raw CD values from the refinement optimizer. Random-init success counts candidates reaching CD < 0.1 across 60 trials per family.

Family	Random Best CD	Random Success
4-bar	5.73×10^{-4}	4/60 (7%)
Stephenson-I	7.19×10^{-2}	1/60 (2%)
Stephenson-III	6.27×10^{-1}	0/60 (0%)
Watt-II	5.26	0/60 (0%)
Watt-I	29.8	0/60 (0%)

4.2.3 Initialization Is the Bottleneck

To isolate where the pipeline breaks down, Table 6 compares the best Chamfer distance among the 85 decoded candidates before and after **AD-L-BFGS** refinement for each query. The reduction factors confirm that the optimizer is working in every case—it reduces CD by one to five orders of magnitude across all ten queries. The difference between success and failure is not the optimizer's effort but its starting point.

The two Watt-I queries (Q8, Q9) provide the clearest evidence. Both target the same topology type (Watt1T1A1), yet the decode stage produces dramatically different initializations: Q8

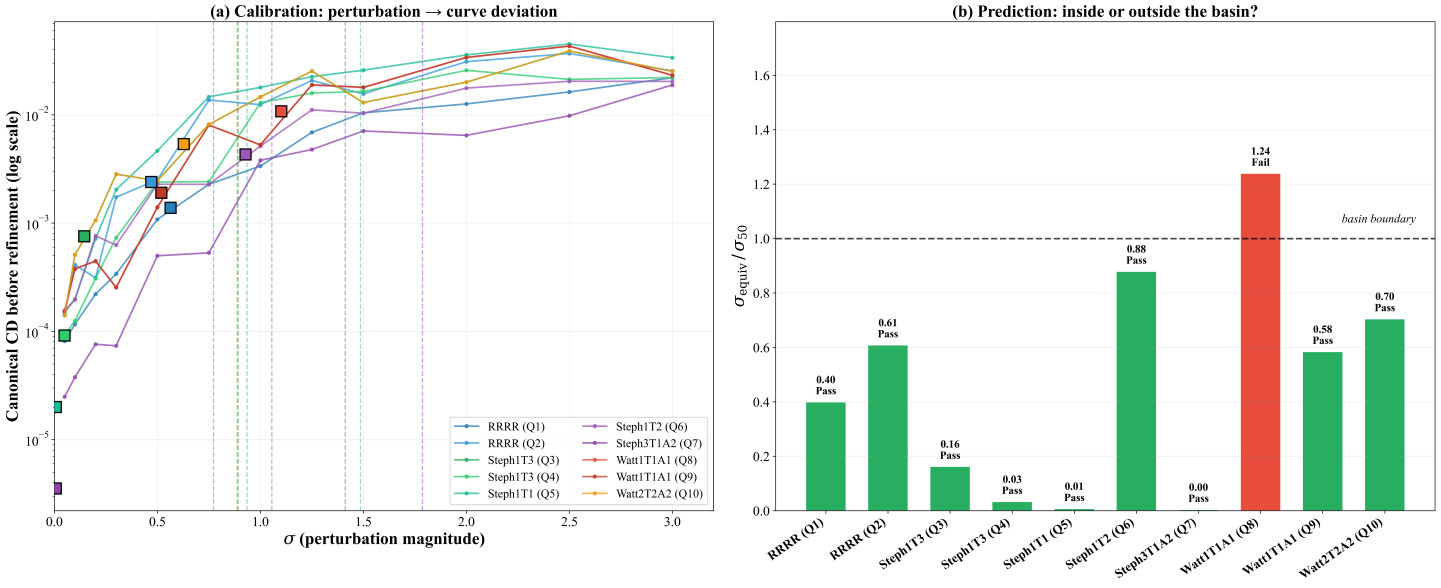

FIGURE 8: Basin-pipeline bridge. Each type's calibration curve is built from 240 basin-study trials (20 ground-truth instances × 12 perturbation magnitudes). (a) Per-type mapping from perturbation magnitude σ to canonical-frame CD (basin study). Square markers show where each pipeline candidate falls on this curve, giving its σ_{equiv}. Vertical dashed lines mark σ_{50} for each type. (b) Ratio $\sigma_{\text{equiv}}/\sigma_{50}$ per query. All nine successful queries (green) fall below 1; the single failure, Q8 Watt-I (red), exceeds 1. Table 5 shows the results in values.

TABLE 5: Basin-pipeline bridge. For each pipeline query, the best candidate's pre-refinement canonical CD is translated to an equivalent perturbation magnitude σ_{equiv} and compared to the type's 50%-success radius σ_{50}. Q# corresponds to Table 3.

Q#	GT Type	Best Type	Canon CD	σ_{equiv}	σ_{50}	$\sigma_{\text{equiv}}/\sigma_{50}$	Predicted	Actual
1	RRRR	Watt2T1A2	1.39×10^{-3}	0.56	1.41	0.40	Success	Success
2	RRRR	Watt1T1A1	2.40×10^{-3}	0.47	0.77	0.61	Success	Success
3	Steph1T3	Watt1T3A1	7.6×10^{-4}	0.14	0.89	0.16	Success	Success
4	Steph1T3	Steph1T3	9.2×10^{-5}	0.05	1.49	0.03	Success	Success
5	Steph1T1	Steph1T1	2.0×10^{-5}	0.007	0.94	0.007	Success	Success
6	Steph1T2	Steph3T2A1	4.31×10^{-3}	0.93	1.06	0.88	Success	Success
7	Steph3T1A2	Steph3T1A2	3.5×10^{-6}	0.007	1.79	0.004	Success	Success
8	Watt1T1A1	Watt1T2A1	1.08×10^{-2}	1.10	0.89	1.24	Fail	Fail
9	Watt1T1A1	Watt1T2A1	1.91×10^{-3}	0.52	0.89	0.58	Success	Success
10	Watt2T2A2	Watt1T3A1	5.38×10^{-3}	0.63	0.89	0.70	Success	Success

starts at 46.3, while Q9 starts at 1.43—a 32× gap. The optimizer applies comparable effort in both cases (reduction factors of 11× and 366×), but Q8 starts too far out. Among the nine successful queries, the best decoded candidate starts with an initial CD between 1.19×10^{-3} and 4.11. For Q8, the initial CD is 46.3—an order of magnitude larger than any success.

The bottleneck is therefore the retrieve-decode stage. When the target curve has close neighbors in the latent space, the decoder produces candidates that the refiner can drive to high precision. When the target is too isolated—as in Q8—no decoded candidate is close enough for local optimization to bridge the gap. This diagnosis points to a concrete path forward: the analytical gradients produced by the differentiable simulator can be propagated through the full decode-simulate-loss chain, enabling

TABLE 6: Initialization quality per query. "Best Initial CD" is the lowest Chamfer distance across the 85 decoded candidates before refinement. "Best Final CD" is the lowest after refinement. Both are measured in the same coordinate frame as the basin study (Section 4.1). Q# corresponds to Table 3.

Q#	GT Type	Best Initial CD	Best Final CD	Reduction
1	RRRR	3.88	6.22×10^{-6}	$\sim 6 \times 10^5$
2	RRRR	2.89	1.83×10^{-4}	$\sim 2 \times 10^4$
3	Steph1T3	1.16	4.38×10^{-3}	~ 265
4	Steph1T3	4.40×10^{-3}	2.06×10^{-5}	~ 214
5	Steph1T1	6.28×10^{-3}	2.71×10^{-4}	~ 23
6	Steph1T2	4.11	1.80×10^{-2}	~ 229
7	Steph3T1A2	1.19×10^{-3}	1.44×10^{-4}	~ 8
8	Watt1T1A1	46.3	4.12	~ 11
9	Watt1T1A1	1.43	3.92×10^{-3}	~ 366
10	Watt2T2A2	7.56×10^{-3}	9.51×10^{-5}	~ 80

the decoder to be trained with a physics-informed loss that produces closer initializations.

5 Summary and Conclusions

We developed a decomposition-aware differentiable simulator that uses pebble-game-based decomposition to reduce the forward constraint solve to smaller block operations, and implemented it as a PyTorch computation graph to enable exact reverse-mode refinement gradients. The experimental evaluation establishes three findings.

1. Speed and capability. **AD-L-BFGS** is 3.34× faster than **FD-LM** across all seventeen topologies, and at CD < 0.001 succeeds on five times as many trials where **FD-LM** fails—a qualitative capability gap, not merely a speedup.

2. Basin geometry is optimizer-independent. Both optimizers recover the same aggregate basin radius ($\sigma = 0.75$) and per-type basin widths, establishing that the convergence basin is a property of mechanism topology, not of the optimizer. The practical implication is that database sampling density should be allocated by topology, with denser coverage for types with narrower measured basins.

3. Basin width predicts pipeline success. The $\sigma_{\text{equiv}}/\sigma_{50}$ criterion correctly predicts all ten pipeline outcomes before refinement is run, providing a practical diagnostic of initialization quality. The random-initialization baseline converges in only 1.7% of trials, confirming that the retrieve-decode stage is not optional.

Limitations. The ten-query evaluation validates the basin predictions but is not a comprehensive benchmark. The VAE and LLaMA decoder were trained on the full database without a held-out validation set, so pipeline accuracy figures may be slightly optimistic; the basin-study results are unaffected as they use ground-truth perturbations with no learned component.

Future work. The differentiable simulator's gradients can be propagated through the full decode-simulate-loss chain to train the decoder with a physics-informed loss, directly addressing the initialization bottleneck identified here. We also plan to extend the basin analysis to higher-DOF mechanisms and evaluate on a formal held-out benchmark.

REFERENCES

[1] McCarthy, J. M. and Soh, G. S., 2010, Geometric Design of Linkages, Interdisciplinary Applied Mathematics, Springer, 2nd edition, doi:10.1007/978-1-4419-7892-9.

[2] García de Jalón, J. and Bayo, E., 1994, Kinematic and Dynamic Simulation of Multibody Systems: The Real-Time Challenge, Springer-Verlag, doi:10.1007/978-1-4612-2600-0.

[3] Nurizada, A., Lyu, Z., and Purwar, A., 2025, "Path Generative Model Based on Conditional β-Variational Auto Encoder for Four-Bar Mechanism Design", Journal of Mechanisms and Robotics, **17(6)**, p. 061004, doi:10.1115/1.4067169.

[4] Pan, Z., Liu, M., Gao, X., and Manocha, D., 2023, "Joint Search of Optimal Topology and Trajectory for Planar Linkages", The International Journal of Robotics Research, **42(4–5)**, pp. 176–195, doi:10.1177/02783649211069156.

[5] Hoskins, J. C. and Kramer, G. A., 1993, "Synthesis of Mechanical Linkages Using Artificial Neural Networks and Optimization", Proceedings of the IEEE International Conference on Neural Networks, San Francisco, CA, USA, pp. 822–J, doi:10.1109/ICNN.1993.298663.

[6] Vasiliu, A. and Yannou, B., 2001, "Dimensional synthesis of planar mechanisms using neural networks: application to path generator linkages", Mechanism and Machine Theory, **36(2)**, pp. 299–310, doi:10.1016/S0094-114X(00)00037-9.

[7] Xie, J. and Chen, Y., 2007, "Application Backpropagation Neural Network to Synthesis of Whole Cycle Motion Generation Mechanism", Proceedings of the Twelfth World Congress in Mechanism and Machine Science, Besançon, France.

[8] Galán-Marín, G., Alonso, F. J., and Del Castillo, J. M., 2009, "Shape Optimization for Path Synthesis of Crank-Rocker Mechanisms Using a Wavelet-Based Neural Net-

work", Mechanism and Machine Theory, **44(6)**, pp. 1132–1143.

[9] Ahmadi, B., Nariman-zadeh, N., and Jamali, A., 2017, "Path Synthesis of Four-Bar Mechanisms Using Synergy of Polynomial Neural Network and Stackelberg Game Theory", Engineering Optimization, **49(6)**, pp. 932–947.

[10] Khan, N., Ullah, I., and Al-Grafi, M., 2015, "Dimensional Synthesis of Mechanical Linkages Using Artificial Neural Networks and Fourier Descriptors", Mechanical Sciences, **6(1)**, pp. 29–34.

[11] Chen, C.-H., 2023, "Application of Multiple Deep Neural Networks to Multi-Solution Synthesis of Linkage Mechanisms", Machines, **11**, p. 1018, doi:10.3390/machines11111018, URL https://doi.org/10.3390/machines11111018.

[12] Kapsalyamov, A., Hussain, S., Brown, N. A., Goecke, R., Hayat, M., and Jamwal, P. K., 2023, "Synthesis of a Six-Bar Mechanism for Generating Knee and Ankle Motion Trajectories Using Deep Generative Neural Network", Engineering Applications of Artificial Intelligence, **117(1)**, p. 105500.

[13] Deshpande, S. and Purwar, A., 2020, "An Image-Based Approach to Variational Path Synthesis of Linkages", ASME Journal of Computing and Information Science in Engineering, **21(2)**, p. 021005, doi:10.1115/1.4048422.

[14] Nurizada, A., Dhaipule, R., Lyu, Z., and Purwar, A., 2024, "A Dataset of 3M Single-DOF Planar 4-, 6-, and 8-Bar Linkage Mechanisms With Open and Closed Coupler Curves for Machine Learning-Driven Path Synthesis", Journal of Mechanical Design, **147(4)**, doi:10.1115/1.4067014.

[15] Heyrani Nobari, A., Srivastava, A., Gutfreund, D., and Ahmed, F., 2022, "LINKS: A Dataset of a Hundred Million Planar Linkage Mechanisms for Data-Driven Kinematic Design", Volume 3A: 48th Design Automation Conference (DAC), American Society of Mechanical Engineers, doi:10.1115/detc2022-89798.

[16] Heyrani Nobari, A., Srivastava, A., Gutfreund, D., Xu, K., and Ahmed, F., 2024, "LInK: Learning Joint Representations of Design and Performance Spaces Through Contrastive Learning for Mechanism Synthesis", Transactions on Machine Learning Research, URL https://openreview.net/forum?id=a1MRjOL6WJ.

[17] Deshpande, S. and Purwar, A., 2019, "Computational Creativity Via Assisted Variational Synthesis of Mechanisms Using Deep Generative Models", Journal of Mechanical Design, **141(12)**, p. 121402, doi:10.1115/1.4044396.

[18] Sharma, S. and Purwar, A., 2022, "A Machine Learning Approach to Solve the Alt-Burmester Problem for Synthesis of Defect-Free Spatial Mechanisms", ASME Journal of Computing and Information Science in Engineering, **22(2)**, doi:10.1115/1.4051913.

[19] Yu, S.-C., Chang, Y., and Lee, J.-J., 2022, "A generative model for path synthesis of four-bar linkages via uniform sampling dataset", Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, **237(4)**, pp. 811–829, doi:10.1177/09544062221123700.

[20] Lee, S., Kim, J., and Kang, N., 2024, "Deep Generative Model-Based Synthesis Framework of Four-Bar Linkage Mechanisms with Target Conditions", Journal of Computational Design and Engineering, **11(5)**, pp. 318–332, doi:10.1093/jcde/qwae084.

[21] Bolanos, D., Ataei, M., and Jayaraman, P. K., 2026, "MechaFormer: Sequence Learning for Kinematic Mechanism Design Automation", Proceedings of the AAAI Conference on Artificial Intelligence, volume 40, pp. 19773–19780, doi:10.1609/aaai.v40i24.39059.

[22] Jadhav, Y. and Farimani, A. B., 2026, "LinkD: AutoRegressive Diffusion Model for Mechanical Linkage Synthesis", arXiv preprint.

[23] Vermeer, K., Kuppens, R., and Herder, J., 2018, "Kinematic Synthesis Using Reinforcement Learning", Volume 2A: 44th Design Automation Conference, ASME, doi:10.1115/detc2018-85529.

[24] Fogelson, M. B., Tucker, C., and Cagan, J., 2023, "GCP-HOLO: Generating High-Order Linkage Graphs for Path Synthesis", Journal of Mechanical Design, **145(7)**, p. 073303, doi:10.1115/1.4062147.

[25] Lyu, Z. and Purwar, A., 2025, "Construction, Reduction, and Decomposition of Planar Mechanisms Via an Extended Pebble Game Framework", Journal of Mechanisms and Robotics, **17(12)**, p. 121009, doi:10.1115/1.4069901, URL https://doi.org/10.1115/1.4069901.

[26] Kingma, D. P. and Welling, M., 2014, "Auto-Encoding Variational Bayes", Computing Research Repository, **abs/1312.6114**.

[27] Nurizada, A. and Purwar, A., 2023, "An Invariant Representation of Coupler Curves Using a Variational AutoEncoder: Application to Path Synthesis of Four-Bar Mechanisms", ASME Journal of Computing and Information Science in Engineering, **24(1)**, doi:10.1115/1.4063726.

[28] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I., 2017, "Attention Is All You Need", Advances in Neural Information Processing Systems, volume 30, pp. 5998–6008.

[29] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G., 2023, "LLaMA: Open and Efficient Foundation Language Models", arXiv preprint arXiv:2302.13971.

[30] Nurizada, A., 2025, "A Generative Model Approach to the Path Synthesis of Planar Linkages", Ph.D. thesis, Stony Brook University.

[31] Lyu, Z. and Purwar, A., 2024, "Deep Learning Conceptual Design of Sit-to-Stand Parallel Motion Six-Bar Mechanisms", Journal of Mechanical Design, **147(1)**, p. 013306, doi:10.1115/1.4066036.

[32] Nocedal, J. and Wright, S. J., 2006, Numerical Optimization, Springer, New York, 2nd edition, doi:10.1007/978-0-387-40065-5.

[33] Frosst, N., Papernot, N., and Hinton, G., 2019, "Analyzing and Improving Representations with the Soft Nearest Neighbor Loss", Proceedings of the 36th International Conference on Machine Learning, PMLR, volume 97 of *Proceedings of Machine Learning Research*, pp. 2012–2020, URL https://proceedings.mlr.press/v97/frosst19a.html.

[34] Laman, G., 1970, "On graphs and rigidity of plane skeletal structures", Journal of Engineering Mathematics, **4**, p. 331–340, doi:10.1007/bf01534980.

[35] Jacobs, D. J. and Thorpe, M. F., 1995, "Generic rigidity percolation: The pebble game", Physical Review Letters, **75(22)**, p. 4051–4054, doi:10.1103/physrevlett.75.4051.

[36] Lee, A. and Streinu, I., 2008, "Pebble game algorithms and sparse graphs", Discrete Mathematics, **308(8)**, pp. 1425–1437, doi:10.1016/j.disc.2007.07.104.

[37] Shai, O., Sljoka, A., and Whiteley, W., 2013, "Directed graphs, decompositions, and spatial linkages", Discrete Applied Mathematics, **161(18)**, pp. 3028–3047, doi:https://doi.org/10.1016/j.dam.2013.06.004, URL https://www.sciencedirect.com/science/article/pii/S0166218X13002813.

[38] Gogu, G., 2005, "Chebychev–Grübler–Kutzbach's criterion for mobility calculation of multi-loop mechanisms revisited via theory of linear transformations", European Journal of Mechanics - A/Solids, **24(3)**, pp. 427–441, doi:https://doi.org/10.1016/j.euromechsol.2004.12.003.

[39] Lyu, Z. and Purwar, A., 2024, "Point-based models: A unified approach for geometric constraint analysis of Planar N-bar mechanisms with Rotary, sliding, and rolling joints", ASME Journal of Mechanisms and Robotics, p. 1–9, doi:10.1115/1.4066963.

[40] Ait-Aoudia, S. and Foufou, S., 2010, "A 2D Geometric Constraint Solver Using a Graph Reduction Method", Advances in Engineering Software, **41**, pp. 1187–1194, doi:10.1016/j.advengsoft.2010.07.008.

[41] Lyu, Z., Purwar, A., and Liao, W., 2023, "A Unified Real-Time Motion Generation Algorithm for Approximate Position Analysis of Planar N-Bar Mechanisms", ASME Journal of Mechanical Design, **146(6)**, doi:10.1115/1.4064132.

[42] Diestel, R., 2017, Graph Theory, volume 173 of *Graduate Texts in Mathematics*, Springer Berlin, Heidelberg, 5th edition, doi:10.1007/978-3-662-53622-3.

[43] Sljoka, A., Shai, O., and Whiteley, W., 2011, "Checking mobility and decomposition of linkages via Pebble Game Algorithm", Volume 6: 35th Mechanisms and Robotics Conference, Parts A and B, doi:10.1115/detc2011-48340.

[44] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E. Z., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S., 2019, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", Advances in Neural Information Processing Systems, Curran Associates, Inc., volume 32, pp. 8024–8035.

[45] Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q., 2018, "JAX: composable transformations of Python+NumPy programs", URL http://github.com/jax-ml/jax.

[46] Hidalgo, M. R. and Joan-Arinyo, R., 2017, "A Henneberg-based algorithm for generating tree-decomposable minimally rigid graphs", Journal of Symbolic Computation, **79**, pp. 232–248, doi:https://doi.org/10.1016/j.jsc.2016.02.006, URL https://www.sciencedirect.com/science/article/pii/S0747717116000158.

[47] Simoulin, A. and Crabbé, B., 2021, "How Many Layers and Why? An Analysis of the Model Depth in Transformers", J. Kabbara, H. Lin, A. Paullada, and J. Vamvas, eds., Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: Student Research Workshop, Association for Computational Linguistics, Online, pp. 221–228, doi:10.18653/v1/2021.acl-srw.23, URL https://aclanthology.org/2021.acl-srw.23/.

[48] Lin, T., Wang, Y., Liu, X., and Qiu, X., 2022, "A survey of transformers", AI Open, **3**, pp. 111–132, doi:10.1016/j.aiopen.2022.10.001, URL https://www.sciencedirect.com/science/article/pii/S2666651022000146.

[49] Petty, J., Steenkiste, S., Dasgupta, I., Sha, F., Garrette, D., and Linzen, T., 2024, "The Impact of Depth on Compositional Generalization in Transformer Language Models", K. Duh, H. Gomez, and S. Bethard, eds., Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Mexico City, Mexico, pp. 7239–7252, doi:10.18653/v1/2024.naacl-long.402, URL https://aclanthology.org/2024.naacl-long.402/.

[50] Moré, J. J., 1978, "The Levenberg-Marquardt algorithm: Implementation and theory", G. A. Watson, ed., Numerical Analysis, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 105–116.

[51] Efron, B. and Tibshirani, R. J., 1993, An Introduction to the Bootstrap, Chapman and Hall/CRC, New York.

[52] Huttenlocher, D. P., Klanderman, G. A., and Rucklidge, W. J., 1993, "Comparing Images Using the Hausdorff Distance", IEEE Transactions on Pattern Analysis and Ma-

chine Intelligence, **15(9)**, pp. 850–863.

